

---

EMPLOYMENT MANAGEMENT SYSTEM

# Errors, Flows, Glossary

---

Status codes, all 18 flows, and what is still missing

ERROR HANDLING AND REFERENCE • 2 OCTOBER 2026

---

## Contents

---

|    |                                                             |    |
|----|-------------------------------------------------------------|----|
| 01 | 1. <code>ErrorResponse</code> - one shape for every failure | 3  |
| 02 | 2. The handler table                                        | 3  |
| 03 | 3. The 21 endpoints, grouped into 18 flows                  | 5  |
| 04 | 4. Where every status code comes from                       | 6  |
| 05 | 5. Known gaps, in priority order                            | 7  |
| 06 | 6. Glossary                                                 | 9  |
| 07 | 7. Where to go next                                         | 10 |

The last document: how failures are reported, the full list of flows, every term defined, and an honest list of what is still missing.

---

## 1. ErrorResponse - one shape for every failure

```
public class ErrorResponse {
    private int status;
    private String message;
    private LocalDateTime timeStamp;
    private String path;           // which endpoint failed
    private Map<String,String> errors; // field-level validation errors
}
```

Example (400):

```
{
  "status": 400,
  "message": "Validation failed: 1 error(s)",
  "timeStamp": "2026-01-15T10:22:31.482",
  "path": "/api/auth/register",
  "errors": { "password": "Password size must be minimum of 6" }
}
```

Every response is built by one private helper, so the shape is guaranteed:

```
private ResponseEntity<ErrorResponse> buildResponse(HttpStatus status, String message,
    HttpServletRequest request, Map<String,String> errors){
    ErrorResponse e = new ErrorResponse(status.value(), message, LocalDateTime.now(),
        request.getRequestURI(), errors);
    return new ResponseEntity<>(e, status);
}
```

`request.getRequestURI()` is why every error knows its own endpoint - handy when debugging. `errors` is `null` for non-validation errors, which serialises as `"errors": null` rather than being omitted.

---

## 2. The handler table

`@RestControllerAdvice` on `GlobalExceptionHandler` means these apply to **every** controller without any per-controller code.

| Thrown                                       | Status | Body message                                            |
|----------------------------------------------|--------|---------------------------------------------------------|
| <code>ResourceNotFoundException</code>       | 404    | the exception's own message                             |
| <code>InvalidInputException</code>           | 400    | the exception's own message                             |
| <code>MethodArgumentNotValidException</code> | 400    | "Validation failed: N error(s)" + per-field map         |
| <code>DuplicateResourceException</code>      | 409    | the exception's own message                             |
| <code>HttpMessageNotReadableException</code> | 400    | "Malformed request body"                                |
| <code>DataIntegrityViolationException</code> | 409    | "Database Constraint Violation: <root cause>"           |
| <code>AccessDeniedException</code>           | 403    | "Forbidden: insufficient permissions"                   |
| <code>BadCredentialsException</code>         | 401    | "Invalid username or password"                          |
| anything else ( <code>Exception</code> )     | 500    | "An unexpected error occurred. Please try again later." |

The generic `500` handler logs the stack trace ( `log.error("Unexpected error at {}: ", request.getRequestURI(), ex)` ) and returns a **fixed** message. That is intentional: never echo an exception message to a client.

### Three custom exceptions, all identical

```
public class ResourceNotFoundException extends RuntimeException {
    public ResourceNotFoundException(String message){ super(message); }
}
```

JAVA

`ResourceNotFoundException`, `InvalidInputException` and `DuplicateResourceException` are byte-for-byte the same shape - each exists so the **handler** can map it to a different status. Because they extend `RuntimeException`, no `throws` clause is needed anywhere and no method is forced to declare them.

The naming convention across the codebase:

| Exception                               | Used for                        | Status |
|-----------------------------------------|---------------------------------|--------|
| <code>ResourceNotFoundException</code>  | "that record does not exist"    | 404    |
| <code>InvalidInputException</code>      | "you sent something impossible" | 400    |
| <code>DuplicateResourceException</code> | "it already exists"             | 409    |

**Inconsistency to know about:** attendance uses `InvalidInputException("Employee not found")`, which produces **400** where the other features produce **404** for the same situation.

### Errors that bypass the advice

The two security responses are written by `SecurityConfig`, not by the advice, because they are produced by the servlet filter chain *before* any controller:

| Response         | Written by                                 | Status | Body                                                                               |
|------------------|--------------------------------------------|--------|------------------------------------------------------------------------------------|
| no/invalid token | <code>authenticationEntryPo<br/>int</code> | 401    | <code>{"status":401,"message":"Unauthorized: invalid or<br/>missing token"}</code> |
| missing role     | <code>accessDeniedHandler</code>           | 403    | <code>{"status":403,"message":"Forbidden: insufficient<br/>permissions"}</code>    |

They have a **different, much smaller body shape** - no `timestamp`, no `path`. `api()` only reads `message`, so the UI behaves identically, but if you ever assert on the error shape in a test you must handle both forms.

Also missing from the advice: `HttpMediaTypeNotSupportedException` (wrong `Content-Type`), `MethodArgumentTypeMismatchException` ( `/employees/abc` ), and `MissingServletRequestParameterException` (missing required param). All three fall into the generic `500` handler, which is why a malformed request returns `500` rather than `400` / ``415`. Those are genuine gaps.

### 3. The 21 endpoints, grouped into 18 flows

A **flow** is what a user accomplishes. This is the map of the whole application.

#### Auth (4 flows, 3 endpoints)

| # | Flow                             | Steps                                                                                                                        | Endpoints |
|---|----------------------------------|------------------------------------------------------------------------------------------------------------------------------|-----------|
| 1 | <b>Log in</b>                    | type credentials -> <code>POST /login</code> -> store token -> <code>GET /me</code> -> navigate <code>/</code>               | 2         |
| 2 | <b>Register</b>                  | type username/password/employee email -> <code>POST /register</code> -> token -> <code>/me</code> -> navigate <code>/</code> | 2         |
| 3 | <b>Restore session on reload</b> | token found -> <code>GET /me</code> -> render app; or <code>401</code> -> logout -> <code>/login</code>                      | 1         |
| 4 | <b>Log out</b>                   | click Logout -> clear query cache -> clear token -> navigate <code>/login</code>                                             | 0         |

#### Employees (5 flows, 7 endpoints)

| # | Flow                   | Steps                                                                                                                      | Endpoints |
|---|------------------------|----------------------------------------------------------------------------------------------------------------------------|-----------|
| 5 | <b>Browse the list</b> | open <code>/employees</code> -> <code>GET /employees/paged</code> -> Prev/Next -> <code>GET /employees/paged?page=n</code> | 1         |
| 6 | <b>Search by name</b>  | type name -> Submit -> <code>GET /employees/search?name=</code>                                                            | 1         |
| 7 | <b>Create</b>          | Add Employee -> modal -> choose department ( <code>GET /departments</code> ) -> <code>POST /employees</code> -> invalidate | 2         |
| 8 | <b>Update</b>          | Edit -> modal -> <code>PATCH /employees/{id}</code> -> invalidate                                                          | 1         |

| # | Flow          | Steps                                                                        | Endpoints |
|---|---------------|------------------------------------------------------------------------------|-----------|
| 9 | <b>Delete</b> | Delete -> confirm -> <code>DELETE /employees/{id}</code> (204) -> invalidate | 1         |

### Departments (5 flows, 7 endpoints)

| #  | Flow                   | Steps                                                                                 | Endpoints |
|----|------------------------|---------------------------------------------------------------------------------------|-----------|
| 10 | <b>Browse the list</b> | open <code>/departments</code> -> <code>GET /departments/paginated</code> -> paginate | 1         |
| 11 | <b>Create</b>          | Add Department -> modal -> <code>POST /departments</code> -> invalidate               | 1         |
| 12 | <b>Update</b>          | Edit -> modal -> <code>PUT /departments/{id}</code> -> invalidate                     | 1         |
| 13 | <b>Delete</b>          | Delete -> confirm -> <code>DELETE /departments/{id}</code> (204)                      | 1         |
| 14 | <b>View details</b>    | View -> modal opens -> <code>GET /departments/{id}/employees</code> -> table          | 1         |

### Attendance (3 flows, 4 endpoints)

| #  | Flow                   | Steps                                                                                                                                                                                  | Endpoints         |
|----|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------|
| 15 | <b>Check in</b>        | select self (or pick an employee) -> Check In -> <code>POST /check-in</code> -> invalidate <code>attendance</code> , <code>attendance-today</code> , <code>dashboard-attendance</code> | 1 + 1-2 refetches |
| 16 | <b>Check out</b>       | Check Out -> <code>POST /check-out</code> -> status computed -> same invalidations                                                                                                     | 1 + refetches     |
| 17 | <b>View attendance</b> | employees: <code>GET /attendance/me</code> ; managers: filters -> Apply -> <code>POST /attendance/search</code> ; paginate                                                             | 1-2               |

### Dashboard (1 flow, 3 endpoints)

| #  | Flow                    | Steps                                                                                                                                                                    | Endpoints |
|----|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| 18 | <b>See the overview</b> | open <code>/</code> -> <code>GET /employees</code> , <code>GET /departments</code> , <code>POST /attendance/search</code> (empty filter, size 1000) -> count client-side | 3         |

Note flows 7 and 18 issue **3 endpoints for one user goal**. That is the clearest illustration of why "endpoint" and "flow" are different words.

## 4. Where every status code comes from

| Status                   | Trigger in this app                                                                              |
|--------------------------|--------------------------------------------------------------------------------------------------|
| <code>200 OK</code>      | any successful read, update, check-in, check-out, login                                          |
| <code>201 Created</code> | <code>POST /employees</code> , <code>POST /departments</code> , <code>POST /auth/register</code> |

| Status                    | Trigger in this app                                                                                                                                                        |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 204 No Content            | DELETE /employees/{id}, DELETE /departments/{id} (empty body)                                                                                                              |
| 400 Bad Request           | bean-validation failure, malformed JSON, empty department name, department still has employees, check-in twice, check-out with no check-in, unknown attendance employee id |
| 401 Unauthorized          | missing/expired/invalid token, wrong username or password                                                                                                                  |
| 403 Forbidden             | authenticated but wrong role, or an employee targeting someone else                                                                                                        |
| 404 Not Found             | unknown employee id, unknown department name, unknown email at registration, missing ROLE_EMPLOYEE row                                                                     |
| 409 Conflict              | duplicate username, duplicate email, duplicate department name, DB constraint violation (e.g. deleting an employee who still has a login)                                  |
| 500 Internal Server Error | anything unhandled - and, unfortunately, unsupported content types and unparseable path variables                                                                          |

## 5. Known gaps, in priority order

### Blocking for a fresh install

**1. No admin/manager account can be created.** `DataSeeder` inserts roles, departments, employees and attendance - but **no** `User` rows. Registration always assigns `ROLE_EMPLOYEE`. So on a new database you cannot reach any manager-only screen.

Two ways out:

*Register as an employee, then promote that row:*

```
USE employee_management_system;
INSERT INTO users_roles (user_id, role_id)
SELECT u.id, r.id FROM users u, roles r
WHERE u.username = 'jane' AND r.name = 'ROLE_MANAGER';
```

SQL

*Or insert an account from scratch with a BCrypt hash of the password:*

```
USE employee_management_system;
INSERT INTO users (username, password, enabled, created_at)
VALUES ('admin', '<bcrypt-hash>', TRUE, NOW());

INSERT INTO users_roles (user_id, role_id)
SELECT u.id, r.id FROM users u, roles r
WHERE u.username = 'admin' AND r.name = 'ROLE_ADMIN';
```

SQL

`User` also implements `UserDetails` without `@Transient` on `isEnabled`, and `enabled` is a primitive `boolean`, so leaving it out yields `false` and login fails with "Invalid username or password".

To generate a hash in a Spring Boot project:

```
new BCryptPasswordEncoder().encode("secret123")
```

JAVA

2. **README.md is out of date.** It claims department writes are ADMIN-only (they are A/M), shows the access column as plain `auth`, and - worst - states that *"authentication is temporarily relaxed"* (`anyRequest().permitAll()`). That is no longer true: `anyRequest().authenticated()` with the full `@PreAuthorize` set is live. It also documents the error shape as `{"message", "timestamp"}` instead of the real `ErrorResponse`.

## Correctness issues worth fixing

| Issue                                                                         | Where                                                                                            | Effect                                                                                                |
|-------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| A single date bound is silently ignored                                       | <code>AttendanceServiceImp</code> line 63 (needs both)                                           | picking only "From" or only "To" filters nothing                                                      |
| <code>new Date().toISOString()</code> is UTC                                  | <code>AttendancePage:36</code>                                                                   | in non-UTC timezones the frontend's "today" can differ from the server's <code>LocalDate.now()</code> |
| Negative <code>totalHours</code> for overnight shifts                         | <code>checkOut</code>                                                                            | an overnight shift is reported <code>HALF_DAY</code>                                                  |
| Department referenced by <b>name</b> in employee APIs                         | <code>EmployeeCreateRequest.departmentName</code>                                                | renaming a department breaks clients; an id would be the fix                                          |
| Search is unpagged and unescaped                                              | <code>EmployeeRepository.findByFullNameContainingWithDepartment</code> , <code>http.js:38</code> | unbounded result set; a name with <code>&amp;</code> breaks the query string                          |
| <code>""</code> passes <code>@Size</code> on <code>DepartmentUpdateReq</code> | <code>DepartmentUpdateReq</code>                                                                 | a blank description can overwrite a <code>nullable = false</code> column                              |
| Unsupported content types return <code>500</code>                             | no handler for <code>HttpMediaTypeNotSupportedException</code>                                   | should be <code>415 / 400</code>                                                                      |
| <code>totalHours</code> is write-only                                         | never in <code>AttendanceResponse</code>                                                         | computed, stored, never displayed                                                                     |
| <code>/me</code> has no <code>employee ≠ null</code> guard                    | <code>AttendanceController:47</code>                                                             | an <code>EMPLOYEE</code> account with no employee row would NPE -> <code>500</code>                   |

## Production readiness

| Issue                                             | Fix                   |
|---------------------------------------------------|-----------------------|
| <code>jwt.secret</code> and DB password committed | environment variables |

| Issue                                                                                                                     | Fix                                                                                      |
|---------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| <code>ddl-auto: update</code>                                                                                             | Flyway/Liquibase + <code>validate</code>                                                 |
| <code>show-sql: true</code>                                                                                               | off in production                                                                        |
| 24-hour tokens, no revocation, no refresh                                                                                 | short TTL + refresh token                                                                |
| Public registration                                                                                                       | invite codes or admin-created accounts                                                   |
| No rate limiting on <code>/login</code>                                                                                   | throttling                                                                               |
| Only <code>EmsApplicationTests</code> exists                                                                              | real <code>@WebMvcTest</code> / <code>@SpringBootTest</code> coverage of the role matrix |
| <code>vercel.json</code> points at <code>YOUR-EMS-BACKEND.onrender.com</code>                                             | real backend URL, plus an SPA fallback rewrite                                           |
| Frontend <code>min="0"</code> allows salary <code>0</code> ; backend <code>@DecimalMin(inclusive=false)</code> rejects it | align the two                                                                            |

## 6. Glossary

| Term                                      | Meaning in this project                                                                                                        |
|-------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| <code>Employee</code>                     | an HR record (name, email, salary, hire date, department). <b>Not</b> a login.                                                 |
| <code>User</code>                         | a login account (username, BCrypt password, roles). Links to at most one <code>Employee</code> .                               |
| <code>Role</code>                         | a row in <code>roles</code> ; the string is <code>ROLE_ADMIN</code> / <code>ROLE_MANAGER</code> / <code>ROLE_EMPLOYEE</code> . |
| authority / <code>GrantedAuthority</code> | Spring Security's view of a role, e.g. <code>ROLE_ADMIN</code> . Produced by <code>User.getAuthorities()</code> .              |
| <code>canManage</code>                    | frontend boolean: "is ADMIN or MANAGER". Drives nav, guards and <code>enabled</code> .                                         |
| JWT                                       | a signed, self-contained token: header + claims + HMAC signature. No server session.                                           |
| <code>SecurityContext</code>              | the per-request holder of the authenticated principal.                                                                         |
| principal                                 | the authenticated object - here, the <code>User</code> entity.                                                                 |
| <code>AuthenticationManager</code>        | Spring component that verifies username + password against <code>UserDetailsService</code> + <code>PasswordEncoder</code> .    |
| BCrypt                                    | salted, deliberately slow password hashing.                                                                                    |
| <code>@PreAuthorize</code>                | SpEL-based method authorization; requires <code>@EnableMethodSecurity</code> .                                                 |
| <code>permitAll</code>                    | the URL needs no authentication. It does <b>not</b> mean "public data".                                                        |
| DTO                                       | data transfer object - the request/response shape, decoupled from the entity.                                                  |
| Entity                                    | the <code>@Entity</code> class that maps a database table.                                                                     |

| Term                                              | Meaning in this project                                                                                                                                 |
|---------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| Mapper (MapStruct)                                | compile-time generated Entity<->DTO converter.                                                                                                          |
| <code>Page&lt;T&gt;</code>                        | Spring Data's paginated result: <code>content</code> , <code>totalPages</code> , <code>totalElements</code> , <code>number</code> , <code>size</code> . |
| <code>Pageable</code>                             | the request-side pagination object: <code>page</code> , <code>size</code> , <code>sort</code> .                                                         |
| <code>Specification</code>                        | Spring Data's Criteria-API predicate; attendance composes them with <code>allof</code> .                                                                |
| <code>@Transactional</code>                       | run the method in one DB transaction; roll back on exception.                                                                                           |
| dirty checking                                    | Hibernate auto- <code>UPDATE</code> s a managed entity's changed fields - why update/delete methods never call <code>save</code> .                      |
| <code>FetchType.LAZY</code>                       | load the association on first access, only inside an open transaction.                                                                                  |
| <code>LEFT JOIN FETCH</code>                      | fetch an association in the same query - required to map it safely.                                                                                     |
| <code>CommandLineRunner</code>                    | Spring's startup hook; <code>DataSeeder</code> uses it.                                                                                                 |
| <code>ddl-auto: update</code>                     | create/migrate tables from the entities at startup.                                                                                                     |
| Zustand                                           | client-state store; <code>create</code> , <code>set</code> , <code>get</code> , selector subscriptions.                                                 |
| TanStack Query                                    | server-state library: caching, <code>useQuery</code> , <code>useMutation</code> , invalidation.                                                         |
| query key                                         | the cache identity of a query; a prefix match is used for invalidation.                                                                                 |
| <code>Outlet</code>                               | the placeholder a layout route renders its child route into.                                                                                            |
| <code>&lt;Navigate replace&gt;</code>             | redirect that <b>replaces</b> the history entry, avoiding Back-button loops.                                                                            |
| <code>enabled</code>                              | TanStack Query flag: do not fire this query yet.                                                                                                        |
| <code>placeholderData</code>                      | keep showing the previous data while a new request is in flight.                                                                                        |
| <code>isLoading</code> vs <code>isFetching</code> | "no data yet" vs "a request is running (maybe a refetch)".                                                                                              |
| <code>invalidateQueries</code>                    | mark cache entries stale and refetch the mounted ones.                                                                                                  |
| proxy                                             | Vite's dev-time <code>/api</code> -> <code>localhost:8080</code> forwarding; the reason there is no CORS config.                                        |
| SpEL                                              | Spring Expression Language - the language inside <code>@PreAuthorize</code> strings.                                                                    |
| <code>record</code>                               | immutable Java data class; accessors have no <code>get</code> prefix.                                                                                   |

## 7. Where to go next

- **To run it:** follow `README.md` sections 1-4 for the database and roles, but ignore the "security is relaxed" note at the bottom - it is out of date. Then create a manager account with the SQL in section 5.1.

- **To change a rule:** business rules live in the `*ServiceImp` classes; access rules live in `@PreAuthorize` annotations; validation rules live in the DTOs.
- **To add an endpoint:** controller -> service interface -> service impl -> repository method -> (if new fields) DTO + mapper. Then add the `http.js` wrapper and invalidate the right query key from the mutation.
- **To verify a change:** `.\mvnw.cmd clean package -DskipTests`, `npx oxlint src`, `npm run build`, then re-check the role matrix from `07-security-model.md` section 4 with a manager token and an employee token.